DEEPFAKE DETECTION SYSTEM

A project submitted in partial fulfilment of the requirements

for the award of the degree of

Bachelor of Technology

in

Computer Science and Engineering

Group Members:

Sankalp (12311001)

Madhur Suman (12311002)

Supervised By:

Dr. Md. Arquam

(Assistant Professor)

INDIAN INSTITUTE OF INFORMATION TECHNOLOGY, SONEPAT

131201, HARYANA, INDIA

April 2026

ACKNOWLEDGEMENT

None of this would have come together without the people who nudged us forward when the experiments stalled and the plots looked wrong. We owe a sincere note of thanks to everyone who pitched in.

Dr. Md. Arquam, Assistant Professor in the CSE department at IIIT Sonepat, let us take on this topic and stayed reachable even when his calendar was packed. That steady backing mattered more than we can fit on one page.

Other faculty in CSE also steered us toward better reading on deep learning and multimedia forensics. Their comments in reviews tightened both the code paths we chose and the way we wrote them up.

Last, our families and friends put up with late nights and last-minute runs; we are grateful for that patience.

Sankalp

Madhur Suman

SELF DECLARATION

We hereby state that the work contained in the project titled "Deepfake Detection System" is original. We have followed the standards of project ethics to the best of our abilities. We have acknowledged all the sources of knowledge that we have used in this project.

Sankalp (12311001)

Madhur Suman (12311002)

Department of Computer Science and Engineering,

Indian Institute of Information Technology,

Sonepat - 131201, Haryana, India

CERTIFICATE

This is to certify that Mr. Sankalp and Mr. Madhur Suman have worked on the project entitled "Deepfake Detection System" under my supervision and guidance.

The contents of the project, being submitted to the Department of Computer Science and Engineering, IIIT Sonepat, Haryana, for the award of the degree of B.Tech in Computer Science and Engineering, are original and carried out by the candidates themselves. This project has not been submitted in full or part for the award of any other degree or diploma to this or any other university.

Dr. Md. Arquam,

Supervisor

Department of Computer Science and Engineering,

Indian Institute of Information Technology,

Sonepat, Haryana

ABSTRACT

Project Title: Deepfake Detection System

Submitted by: 1. Sankalp (12311001) 2. Madhur Suman (12311002)

Degree: B.Tech

Project Supervisor: Dr. Md. Arquam

Month and Year: April, 2026

Department: Computer Science and Engineering, IIIT Sonepat, Haryana

We built a detector that listens as well as it looks. Synthetic clips often break down at the boundary between picture and sound—lip timing slips, the voice does not quite match the face, or a splice shows up only when both tracks are compared. A ViT-B/16 backbone reads sixteen sampled frames; a compact four-layer CNN turns mel spectrograms into tokens; a Multimodal Bottleneck Transformer (MBT) lets those streams talk without full cross-attention everywhere.

Training is split into two stages. First, on unlabelled VoxCeleb video, we run three self-supervised losses together: MoCo-style cross-modal contrastive learning with a memory bank, a sync-vs-desync classifier, and masked reconstruction for both modalities. Second, we load those encoders into a binary head and fine-tune on FakeAVCeleb. Everything runs in PyTorch with automatic mixed precision. On FakeAVCeleb validation we reached 85.2% accuracy, 86.42% precision, 75.6% recall, and an F1 of 80.1%.

TABLE OF CONTENTS

LIST OF ABBREVIATIONS

ABBREVIATION	FULL FORM
AI	Artificial Intelligence
ML	Machine Learning
DL	Deep Learning
CNN	Convolutional Neural Network
ViT	Vision Transformer
MBT	Multimodal Bottleneck Transformer
GAN	Generative Adversarial Network
SSL	Self-Supervised Learning
MoCo	Momentum Contrast
AV	Audio-Visual
GPU	Graphics Processing Unit
CPU	Central Processing Unit
API	Application Programming Interface
BCE	Binary Cross-Entropy
AMP	Automatic Mixed Precision
MLP	Multi-Layer Perceptron
GELU	Gaussian Error Linear Unit
InfoNCE	Information Noise Contrastive Estimation
FPS	Frames Per Second

Chapter 1

Introduction

1.1 INTRODUCTION

Cheap generative models can now fabricate faces, voices, and short clips that look fine at first glance. People call the worst of these deepfakes—often built with GANs, diffusion nets, or heavy autoencoder stacks. A creator can paste a face, mimic a voice, or nudge expressions until the clip passes casual viewing. The same line of research that produced demos for papers has spilled into scams, impersonation, and coordinated disinformation, so trust in ordinary video is shakier than it was even five years ago.

Manual inspection does not scale, and old fingerprint tricks (compression quirks, blockiness) miss many modern fakes because the synthesis is too clean. We still need detectors that run on GPUs people can rent, update when generators change, and flag content before it spreads.

We lean on both modalities on purpose. Plenty of prior work stares at frames alone; we pair RGB sequences with audio because many forgeries show up only in the coupling—mouth shape lagging syllables, a laugh that arrives half a beat late, or speech spliced from another take. A multimodal model can chase those tells.

Pretraining is label-free on real speech video (VoxCeleb), then we snap on a small supervised head for FakeAVCeleb. ViT-B/16 carries the visual side, a four-layer CNN maps mel bins to tokens, and MBT sits in the middle so the streams exchange information through shared bottlenecks instead of all-to-all attention.

1.2 PROBLEM OUTLINE

The literature is large, yet a few pain points keep showing up in practice.

Vision-only detectors chase spatial cues—blurred jawlines, odd pores, lighting that flickers between frames. That can work inside the benchmark they were tuned on, yet a new generator may hide those cues altogether, so pure frame classifiers age quickly.

Labels are scarce and expensive. Public packs cover a slice of methods and scenes, so a model that memorizes one corpus often stumbles outdoors the moment the attack recipe changes.

Audio is frequently dropped even though many manipulations leave the soundtrack misaligned or borrowed from elsewhere. Ignoring the waveform throws away half the story.

Video is heavy: sixteen frames per clip through a ViT is not lightweight, so any serious pipeline must trade resolution, batch size, and wall-clock time against what a lab or a startup can actually afford.

1.3 PROJECT OBJECTIVES

We set out to ship a detector that is multimodal, partly self-supervised, measurable on FakeAVCeleb, and split into modules we can swap later.

- **Cross-modal cues.** Fuse RGB tokens and mel tokens through MBT so timing and appearance are judged jointly rather than in isolation.
- **Self-supervised front end.** Pretrain without fake labels on VoxCeleb using contrastive alignment, sync discrimination, and masked reconstruction, then fine-tune on FakeAVCeleb with a thin classifier head.
- **Benchmarks and budget.** Report accuracy, precision, recall, and F1 while staying within reach of a mid-range GPU via AMP and sensible batching.
- **Clean seams.** Keep encoders, fusion, and head in separate classes so we can drop in a stronger ViT or audio model without rewriting the whole repo.

1.4 PROJECT METHODOLOGY

We moved in six concrete steps from papers to checkpoints.

1. Reading and stack pick. After skimming forgery surveys and multimodal fusion work, we settled on PyTorch, ViT-B/16 for frames, and MBT-style bottlenecks for fusion.

2. Data pulls. VoxCeleb feeds SSL; FakeAVCeleb supplies labelled real/fake pairs. Scripts sample sixteen frames, crop centres, apply ImageNet stats, and build log-mel spectrograms at 16 kHz.

3. Code layout. VideoViTEncoder wraps torchvision ViT with temporal position codes; AudioTokenEncoder is a four-layer CNN into 768-D; AudioVisualFusionEngine stacks MBT layers with four shared tokens.

4. SSL loop. Contrastive InfoNCE with a memory bank, sync negatives (swapped clips and time shifts), and masked blocks on both modalities. ssl_pretrain.py is set to 50 epochs (AdamW, AMP, grad clip 1.0); Table 5.1 / Figure 5.1 show an 8-epoch snapshot from our logged run.

5. Fine-tune. Load SSL weights into DeepfakeClassifier, attach a three-layer MLP head, and use a lower LR on pretrained blocks than on the head.

6. Numbers. Track loss curves in TensorBoard, dump PNG dashboards for finetune metrics, and score validation with accuracy / precision / recall / F1.

1.5 SCOPE OF PROJECT WORK

In scope: end-to-end training code, not a polished consumer app.

- **Dual encoders.** Sixteen RGB frames and matching mel tensors, each mapped to 768-D sequences with temporal embeddings.
- **Fusion.** Four MBT layers, four bottleneck tokens—information passes through the narrow slots instead of dense audio-visual attention.
- **Pretrain stack.** SSL with flips, colour jitter, light blur, gain noise, SpecAugment masks, MoCo bank, sync games, and masked reconstruction.
- **Supervised wrap-up.** Finetune with 80/20 split, BCE-with-logits, cosine schedule, AMP, and plots for loss and classification metrics.

1.6 LIMITATIONS

- **GPU memory.** SSL with full ViT forward passes wants VRAM; 16 GB is a rough floor and 24 GB (3090/4090 class) is more comfortable.
- **Domain shift.** VoxCeleb is interview-heavy; FakeAVCeleb reflects certain synthesis recipes—new attacks may still slip past.
- **Missing sound.** No audio means our design loses half its signal; we do not ship a visual-only fallback path in code.
- **Latency.** Training scripts batch offline clips; live moderation would need quantisation, distillation, or smaller backbones.

We did not run targeted adversarial evasion tests; a patient attacker could optimise against our logits, which is left for future work.

1.7 ORGANIZATION OF THE PROJECT

Five chapters walk from motivation to numbers.

- **Chapter 1** Motivation, goals, how we worked, what we included, and where we stopped.
- **Chapter 2** Python stack and the CLI workflow for download → preprocess → train.
- **Chapter 3** Stages from MP4 on disk to logits out of the classifier.
- **Chapter 4** Encoders, fusion, losses, loading checkpoints, and augmentation—with code fragments.
- **Chapter 5** Curves, tables, takeaway, and what we would try next.

1.8 SUMMARY

Chapter 1 set the scene: forgeries are easier to make, harder to eyeball, and often visible only when audio and video disagree. We listed what blocks deployment—narrow vision cues, label shortage, ignored waveforms, compute—and described how we train in two phases with MBT in the middle. Later chapters unpack scripts, tensors, and metrics.

Chapter 2

Application and User Interface

2.1 TECHNOLOGIES EMPLOYED

Everything is Python-first; the list below is the glue we actually import day to day.

- **Runtime.** Python 3.8+ for models, dataloaders, and glue scripts.
- **Core DL.** PyTorch 2.x modules (nn, optim, amp) for forward/backward and CUDA kernels.
- **Vision.** TorchVision: ViT-B/16 weights plus read_video and the usual resize/crop/normalise helpers.
- **Audio.** TorchAudio: resample to 16 kHz, MelSpectrogram, AmplitudeToDB.
- **Plots.** Matplotlib for PNG loss boards after each phase.
- **Live scalars.** TensorBoard while SSL runs long.
- **Progress.** tqdm on every epoch loop so we see ETA when a run stretches overnight.
- **Downloads.** gdown and requests for scripted dataset fetches.

2.2 USER INTERFACE DESCRIPTION

There is no GUI—only terminals, flags in code, and saved weights. That keeps the footprint small for a coursework repo.

Training Pipeline Interface

Rough order of operations:

1. dataset/pretrain_datset_download.py — pull VoxCeleb, unpack archives.

2. dataset/pretrain_prepare_dataset.py — flatten folders so loaders see a simple list of MP4 paths.

3. dataset/preprocess_dataset.py — bake frames + mel tensors to .pt shards to avoid decoding every epoch.

4. ssl_pretrain.py — run SSL objectives, stream scalars to TensorBoard, stash checkpoints under outputs/.

5. dataset/prepare_finetune_dataset.py — park FakeAVCeleb clips under real/ and fake/ with balanced counts.

6. finetune_deepfake.py — load SSL weights, train the binary head, keep the best F1 checkpoint, write metric PNGs.

Inference Interface

At test time you torch.load the best checkpoint, push tensors through the same preprocessors, apply sigmoid to the logit, and treat scores above 0.5 as fake.

Output Artifacts

Expect loss PNGs, TensorBoard event files, finetune dashboards under metrics/, and .pth checkpoints you can copy elsewhere.

Chapter 3

System and Pipeline Design

3.1 INTRODUCTION TO THE SYSTEM PIPELINE

A clean pipeline saves debugging time: you can unit-test preprocessing without touching fusion, and swap encoders without rewriting dataloaders. Multimodal work doubles the moving parts—waveform and pixels must stay time-aligned.

Data enters as MP4, splits into RGB and audio tensors, passes through separate encoders, meets inside MBT, then lands in a shallow MLP that emits one logit.

3.2 PROJECT SYSTEM DESCRIPTION

Five stages, mapped roughly to folders in the repo.

Stage 1: Data Ingestion and Splitting

DeepfakeDataset walks real/ and fake/, stores (path, label), shuffles, and holds an 80/20 split. DataLoader workers pin memory so batches reach the GPU without stalling the CPU.

Stage 2: Preprocessing

Video and audio split here.

RGB: linspace sixteen indices across the clip, centre-crop to 0.8 of the short side, resize to 224^2, normalise with ImageNet μ/σ.

Audio: mono downmix, 16 kHz resample, 1024 FFT / 512 hop mel with 128 bins, AmplitudeToDB, pad/trim to ninety-four time bins so batches stack cleanly.

Stage 3: Feature Encoding

VideoViTEncoder runs ViT-B/16 per frame, keeps CLS tokens, adds learned temporal positions.

AudioTokenEncoder stacks four conv blocks (frequency shrinks, time mostly preserved), projects to 768-D, adds temporal positions.

Stage 4: Multimodal Fusion

AudioVisualFusionEngine prepends CLS tokens, seeds four bottleneck vectors, and stacks four MBT layers. Each modality self-attends with bottlenecks—not with the other stream directly—then bottlenecks are averaged across modalities before the next layer.

Stage 5: Classification

Concatenate final visual and audio CLS (1536-D), MLP 1536→512→256→1 with GELU + dropout, sigmoid at inference with a 0.5 cut.

AMP wraps training steps; gradients clip at norm 1.0 to stop spikes.

Chapter 4

Model Design and Logic

4.1 DEEP LEARNING MODELS USED

Three trainable pieces: a frame encoder, a spectrogram encoder, and the MBT stack that negotiates between them.

Vision Transformer (ViT-B/16) as Visual Encoder

ViT-B/16 chops each 224^2 frame into 16×16 patches, embeds them, and stacks transformer blocks—see Dosovitskiy et al. for the full recipe.

We start from IMAGENET1K_V1 weights, strip the classifier head for an identity, and emit one 768-D CLS vector per frame. VideoViTEncoder adds temporal position codes across the sixteen CLS outputs so fusion sees order, not just appearance.

CNN-Based Audio Tokenizer as Audio Encoder

Mel maps stay 2-D; the CNN squeezes frequency while leaving time mostly untouched until late pooling, yielding a token per time step.

Four 3×3 conv blocks use GroupNorm + ReLU; early max-pool steps shrink freq and time, later (2,1) pools attack frequency only. Adaptive pooling collapses the last freq axis; a linear + LayerNorm maps 256→768, then temporal positions match the video side.

Multimodal Bottleneck Transformer (MBT) as Fusion Engine

Following Nagrani et al., audio and vision never attend across streams wholesale—they read and write four shared bottleneck tokens instead.

Four MBT layers, each with parallel ViT-style blocks per modality, eight heads, dropout 0.1: bottlenecks append to both sequences, attention runs, bottlenecks merge by averaging, repeat.

4.2 MODEL LOADING

Weights arrive twice: ImageNet for ViT at construction, later SSL checkpoints when the classifier spins up.

Pretrained ViT Loading

VideoViTEncoder pulls torchvision weights like this:

```
weights = models.ViT_B_16_Weights.IMAGENET1K_V1 if pretrained else None
self.vit = models.vit_b_16(weights=weights)
self.vit.heads = nn.Sequential(nn.Identity())
```

Spatial filters start strong; temporal embeddings draw fresh noise ($\sigma \approx 0.02$) because SSL must learn order from video.

SSL Encoder Loading for Fine-Tuning

DeepfakeClassifier accepts either bundled encoder dicts or prefixed state keys—handy when a checkpoint comes from the full SSL module.

```
if os.path.exists(pretrained_path):
    checkpoint = torch.load(pretrained_path, map_location='cpu')
    if 'visual' in checkpoint:
        self.visual_encoder.load_state_dict(checkpoint['visual'], strict=False)
        self.audio_encoder.load_state_dict(checkpoint['audio'], strict=False)
        self.fusion_engine.load_state_dict(checkpoint['fusion'], strict=False)
    else:
        visual_state = {k.replace('visual_encoder.', ''): v
                        for k, v in checkpoint.items()
                        if k.startswith('visual_encoder.')}
```

strict=False tolerates small name shifts between training scripts; missing keys simply stay random. After ssl_pretrain.py finishes, encoders are saved as outputs/ssl_encoders_fused.pth (visual/audio/fusion keys); finetune_deepfake.py points there by default.

4.3 PREDICTION LOGIC

Forward pass is short: encode, fuse, concat CLS, classify. Reference implementation:

```
def forward(self, visual, audio):
    v_tokens = self.visual_encoder(visual)
    a_tokens = self.audio_encoder(audio)
    fusion_out = self.fusion_engine(v_tokens, a_tokens)
    vis_cls = fusion_out["vis_cls"]
    aud_cls = fusion_out["aud_cls"]
    combined = torch.cat([vis_cls, aud_cls], dim=1)
    logits = self.classifier(combined)
    return logits
```

Inputs: B×T×3×224×224 video and matching mel tensor. ViT emits sixteen 768-D tokens; CNN emits a time sequence at 768-D.

MBT returns fused CLS summaries per modality after four layers; we concatenate for a 1536-D joint vector.

Head: GELU MLP, BCE-with-logits in training, sigmoid + 0.5 threshold when evaluating.

4.4 DATA PROCESSING

VisualPreprocessor and AudioPreprocessor share the same math in SSL and fine-tune so tensors line up with whatever the encoders expect.

Visual Preprocessing

Frame sampling uses torch.linspace across the decoded stack:

```
def _sample_frames(self, frames):
    total_frames = frames.shape[0]
    if total_frames == self.num_frames:
        return frames
    indices = torch.linspace(0, total_frames - 1, self.num_frames).long()
    return frames[indices]
```

Then centre-crop, resize, ImageNet normalise—nothing exotic, just consistent.

Audio Preprocessing

Mel + dB scaling; key hyperparameters below.

```
self.mel_transform = torchaudio.transforms.MelSpectrogram(
    sample_rate=sample_rate, n_fft=n_fft,
    hop_length=hop_length, n_mels=n_mels, normalized=True
)
self.amplitude_to_db = torchaudio.transforms.AmplitudeToDB(
    stype='power', top_db=80
)
```

Log-mel matches human loudness perception and tends to exaggerate narrowband glitches. Fixed ninety-four time bins keep batches square.

Batch Preprocessing for SSL

preprocess_dataset.py bakes each clip once to disk; SSLDataset then mmap-reads tensors, which beats decoding MP4 every epoch.

4.5 TRAINING AND INFERENCE

SSL teaches alignment without labels; supervised fine-tune pins the decision boundary on FakeAVCeleb.

Phase 1: Self-Supervised Pretraining

Three losses run jointly on VoxCeleb.

Cross-modal contrastive: project fused CLS tokens to 256-D L2-normalised space, InfoNCE with in-batch negatives plus a 2048-slot MoCo bank (momentum 0.999) and a light variance term so embeddings do not collapse.

Sync task: half the minibatch stays aligned; half is corrupted—30% with foreign audio, 70% with time-shifted audio (15–40 frames). The head learns aligned vs not.

Masked reconstruction: block masks (visual ~85% in 8-token chunks, audio ~80% in 12-bin chunks), smooth L1 only on masked units.

Loss weights: 0.5 contrastive, 1.0 sync, 2.0 audio recon, 3.0 visual recon, 0.1 variance. Fifty epochs, AdamW 1e-4, wd 1e-4, four-step accumulation (effective batch 32), AMP, grad clip 1.0.

Phase 2: Supervised Fine-Tuning

Load SSL weights into DeepfakeClassifier; split parameter groups so pretrained layers move slower:

```
optimizer = torch.optim.AdamW([
    {"params": pretrained_params, "lr": LEARNING_RATE * 0.1},
    {"params": classifier_params, "lr": LEARNING_RATE}
], weight_decay=1e-4)
```

Base LR 1e-4 on the head, 1e-5 on encoders/fusion. finetune_deepfake.py uses 20 epochs, batch size 4, BCE-with-logits, cosine decay, AMP, grad clip 1.0, best-F1 checkpoint kept as outputs_finetune/best_deepfake_detector.pth.

Inference

eval(), identical preprocessing, no gradient:

```
checkpoint = torch.load("outputs_finetune/best_deepfake_detector.pth")
model = DeepfakeClassifier()
model.load_state_dict(checkpoint['model_state_dict'])
model.eval()

with torch.no_grad():
    logits = model(video_tensor, audio_tensor)
    probability = torch.sigmoid(logits)
    is_fake = probability > 0.5
```

Probabilities near one mean aggressive fake estimates; near zero the model leans real.

4.6 ERROR HANDLING

Broken clips should not kill a week-long run.

Data Loading Errors

DeepfakeDataset wraps torchvision.io.read_video in try/except; failures return zero tensors with the right shape so the iterator keeps moving (not ideal for learning, but stable).

```
try:
    video, audio, info = torchvision.io.read_video(video_path, pts_unit='sec')
    ...
except Exception as e:
    print(f"Error loading video {video_path}: {e}")
    return torch.zeros(self.num_frames, 3, self.target_size, self.target_size)
```

Silent audio falls back similarly—better than aborting mid-epoch.

Dataset Validation

If real/ or fake/ is empty, __init__ raises immediately with the folder path in the message.

```
if len(self.samples) == 0:
    raise RuntimeError(f"No videos found in {root_dir}/real or {root_dir}/fake")
```

Model State Validation

VideoViTEncoder asserts a five-dimensional input (B,T,C,H,W) before ViT runs.

```
assert frames.dim() == 5, f"Expected 5D input (B, T, C, H, W), got {frames.dim()}D"
```

Training Stability

Grad clipping, AMP scaler, and the contrastive variance term together curb blow-ups and collapsed embeddings.

4.7 ADDITIONAL LOGIC: DATA AUGMENTATION

SSL augments aggressively; fine-tune sticks to clean tensors so distribution shift stays manageable.

Video Augmentation

VideoAugmentor:

• 50% mirror flip, same flag for all frames in the clip.

• 80% colour jitter (brightness/contrast/saturation 0.6–1.4), shared across frames.

• 10% light Gaussian blur on every frame to mimic soft focus or heavy compression.

Audio Augmentation

AudioAugmentor:

• Random gain 0.7–1.4 half the time.

• Two time masks (≤10 bins each).

• Two frequency masks (≤15 mel channels each).

Fine-tune and inference skip these transforms.

Chapter 5

Result and Conclusion

5.1 RESULTS AND OBSERVATIONS

We watched SSL losses fall, then checked FakeAVCeleb metrics after fine-tune—both stages tell part of the story.

Self-Supervised Pretraining Results

SSL: the script trains up to 50 epochs; the table below matches the eight-epoch checkpoint we used for Figure 5.1 (same run as the loss snapshot in this report). TensorBoard has the full series if you train longer.

Metric	Initial Value	Final Value (Epoch 8)
Total Loss	1.50	0.60
Contrastive Loss	1.40	0.04
Sync Loss	0.24	0.05
Audio Reconstruction Loss	0.19	0.20
Visual Reconstruction Loss	0.06	0.05

Total loss 1.50→0.60; contrastive 1.40→0.04 (pairs line up in embedding space); sync 0.24→0.05; reconstructions hover low—so masking still teaches detail. Figure 5.1 overlays the five curves.

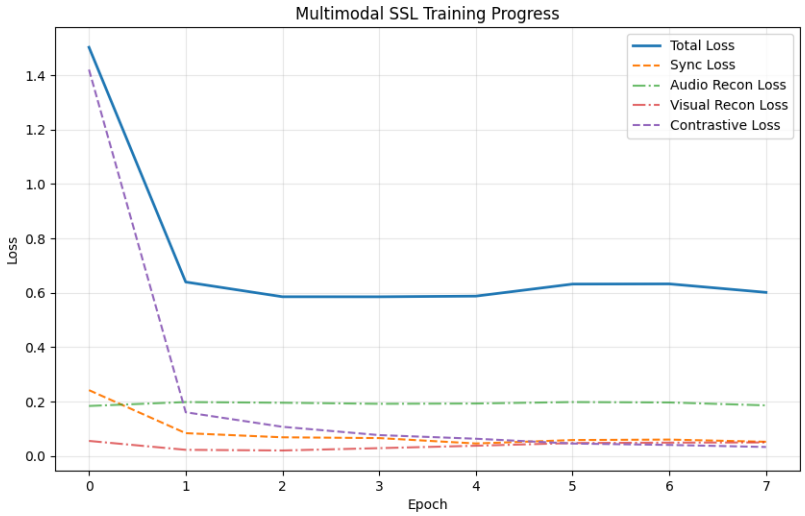


Figure 5.1: Multimodal SSL training progress — per-objective losses (RunMetric snapshot or outputs/loss_curve.png from ssl_pretrain.py)

Fine-Tuning Results

FakeAVCeleb validation scores:

Metric	Score
Accuracy	85.2%
Precision	86.42%
Recall	75.6%
F1 Score	80.1%

85.2% accuracy is solid for the split we used. Precision 86.42% means flagged fakes are usually right; recall 75.6% leaves room—about one in four fakes slips through at the default threshold. F1 80.1% sits between those extremes.

Precision beats recall: the model prefers marking doubtful clips as real, which tracks with a conservative 0.5 cut and class balance. Operators can move the threshold if false negatives hurt more than false positives.

Figure 5.2 stitches loss, accuracy, and PR-F1 across the 20 fine-tuning epochs defined in finetune_deepfake.py.

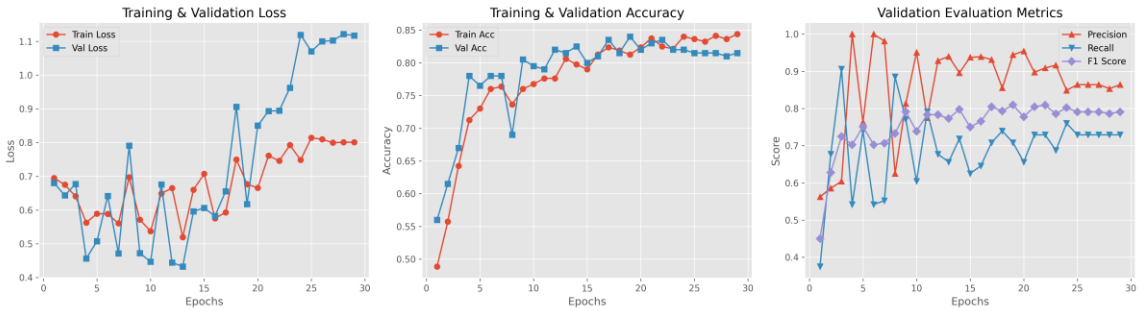


Figure 5.2: Fine-tuning metrics dashboard — loss, accuracy, precision/recall/F1 (metrics/training_metrics_dashboard.png from finetune_deepfake.py, 20 epochs)

Key Observations

Audio+video fusion buys signal we would drop with vision-only SSL. Pretraining buys faster finetune convergence; freezing LR ratios stops catastrophic forgetting in the encoders.

5.2 CONCLUSION

Pairing ViT features, CNN mel tokens, and MBT fusion yielded 85.2% accuracy and 80.1% F1 on FakeAVCeleb under our split—enough to show the idea carries weight.

SSL on real-only VoxCeleb cuts the labelled burden: the encoders already know how speech lines up with faces before fake labels appear.

Modules are split on purpose—swap ViT for a newer variant, drop in an audio transformer, or add side channels without rewriting the repo from scratch.

Bottleneck fusion is not the last word, but it is a workable baseline we can iterate from.

5.3 FUTURE SCOPE

Short list of what we would tackle next.

Ship a thin FastAPI/Flask wrapper so reviewers can upload MP4s instead of editing paths in Python.

Chase real-time: quantise, distil, or shrink backbones so a stream could be scored near frame rate.

Add optical flow, landmarks, or FFT cues as extra inputs—the file layout already invites new tensors.

Benchmark beyond FakeAVCeleb (FaceForensics++, Celeb-DF, DFDC) and study domain shift explicitly.

Explainability (attention rollout, saliency) plus on-device compression matter if anyone deploys this outside a lab.

REFERENCES

[1] Dosovitskiy, A., Beyer, L., Kolesnikov, A., et al. (2021). An Image is Worth 16x16 Words: Transformers for Image Recognition at Scale. In Proceedings of ICLR.

[2] Nagrani, A., Yang, S., Arnab, A., et al. (2021). Attention Bottlenecks for Multimodal Fusion. In Advances in NeurIPS, 34, 14200-14213.

[3] He, K., Fan, H., Wu, Y., Xie, S., & Girshick, R. (2020). Momentum Contrast for Unsupervised Visual Representation Learning. In Proceedings of IEEE/CVF CVPR (pp. 9729-9738).

[4] Khalid, H., Tariq, S., Kim, M., & Woo, S. S. (2021). FakeAVCeleb: A Novel Audio-Video Multimodal Deepfake Dataset. In Proceedings of the NeurIPS Datasets and Benchmarks Track.

[5] Nagrani, A., Chung, J. S., & Zisserman, A. (2017). VoxCeleb: A Large-Scale Speaker Identification Dataset. In Proceedings of Interspeech (pp. 2616-2620).

[6] Park, D. S., Chan, W., Zhang, Y., et al. (2019). SpecAugment: A Simple Data Augmentation Method for Automatic Speech Recognition. In Proceedings of Interspeech (pp. 2613-2617).

[7] Loshchilov, I., & Hutter, F. (2019). Decoupled Weight Decay Regularization. In Proceedings of ICLR.

[8] Micikevicius, P., Narang, S., Alben, J., et al. (2018). Mixed Precision Training. In Proceedings of ICLR.

[9] Touvron, H., Cord, M., Douze, M., et al. (2021). Training Data-Efficient Image Transformers and Distillation Through Attention. In Proceedings of ICML (pp. 10347-10357).

[10] Zi, B., Chang, M., Chen, J., Ma, X., & Jiang, Y. G. (2020). WildDeepfake: A Challenging Real-World Dataset for Deepfake Detection. In Proceedings of ACM MM (pp. 2382-2390).

APPENDIX

Project Repository

GitHub: https://github.com/madhur-suman/DeepDetect2

Project Structure

```
DeepDetect2/
+-- Models/
|   +-- Visual_encoder.py        # ViT-based video encoder
|   +-- audio_encoder.py         # 4-layer CNN audio tokenizer
|   +-- Fusion_Engine.py         # Multimodal Bottleneck Transformer
|   +-- ssl_tasks.py             # SSL pretraining orchestrator
+-- utils/
|   +-- visual_preprocessing.py  # Frame sampling, cropping, normalization
|   +-- audio_preprocessing.py   # Mel spectrogram extraction
|   +-- augmentations.py         # Video and audio augmentation
+-- dataset/
|   +-- pretrain_datset_download.py
|   +-- pretrain_prepare_dataset.py
|   +-- preprocess_dataset.py
|   +-- download_finetune_dataset.py
|   +-- prepare_finetune_dataset.py
+-- ssl_pretrain.py              # Self-supervised pretraining (Phase 1)
+-- finetune_deepfake.py         # Supervised fine-tuning (Phase 2)
+-- outputs/                     # ssl_encoders_fused.pth, loss_curve.png, per-
epoch checkpoints
+-- outputs_finetune/            # Fine-tuned checkpoints
+-- metrics/                     # Training metric plots
+-- README.md
```

Key Dependencies

```
Python >= 3.8
PyTorch >= 2.0
TorchVision
TorchAudio
tqdm
matplotlib
tensorboard
gdown
requests
```

Hardware Used for Training

Component	Specification
GPU	NVIDIA T4 (16 GB VRAM)
RAM	16 GB DDR4
Storage	512 GB SSD
OS	Ubuntu 22.04 LTS

DrillBit

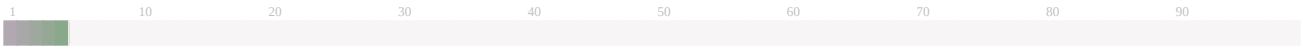
The Report is Generated by DrillBit Plagiarism Detection Software

Submission Information

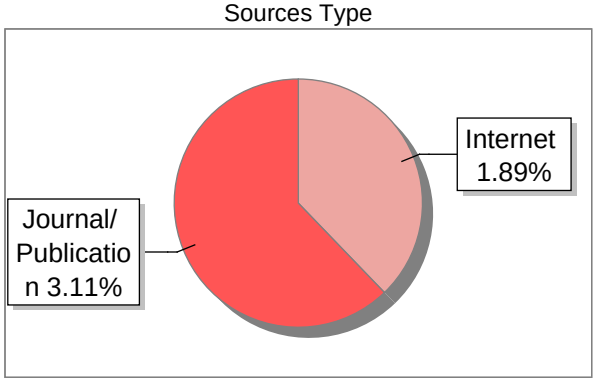
Author Name	Madhur Suman
Title	Deepfake detection systems
Paper/Submission ID	5482781
Submitted by	md.arquam@iiitsonepat.ac.in
Submission Date	2026-04-14 09:52:12
Total Pages, Total Words	30, 4225
Document type	Research Paper

Result Information

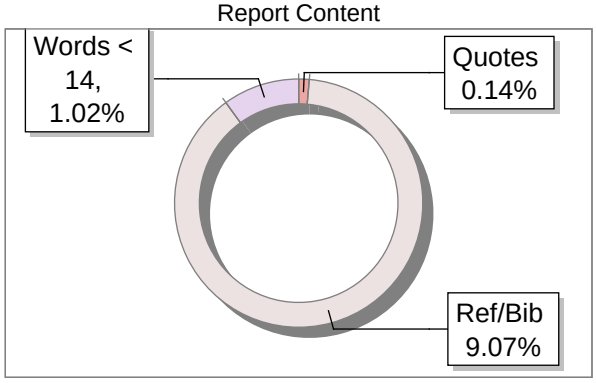
Similarity **5 %**



Sources Type



Report Content



Exclude Information

Quotes	Not Excluded
References/Bibliography	Not Excluded
Source: Excluded < 14 Words	Not Excluded
Excluded Source	**0 %**
Excluded Phrases	Not Excluded

Database Selection

Language	English
Student Papers	Yes
Journals & publishers	Yes
Internet or Web	Yes
Institution Repository	Yes

A Unique QR Code use to View/Download/Share Pdf File

DrillBit

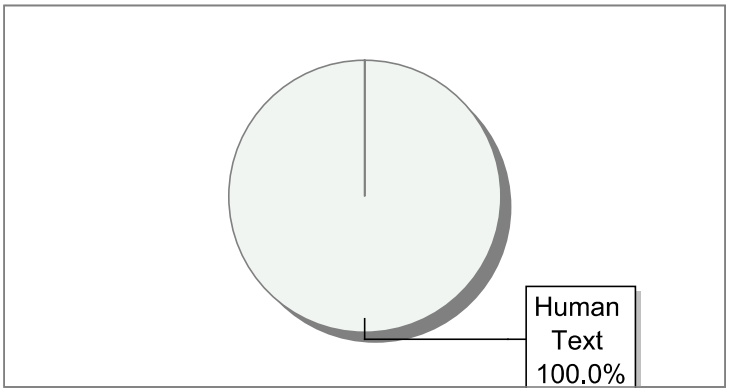
The Report is Generated by DrillBit AI Content Detection Software

Submission Information

Author Name	Madhur Suman
Title	Deepfake detection systems
Paper--Submission ID	5482781
Submitted By	md.arquam@iiitsonepat.ac.in
Submission Date	2026-04-14 09:52:12
Total [Pages,Sentences,Words]	[30, 217, 3692]
Document type	Research Paper
Language	English

Result Information

AI Text: **0 %**



Disclaimer:

* The content detection system employed here is powered by artificial intelligence (AI) technology.
* Its not always accurate and only help to author identify text that might be prepared by a AI tool.
* It is designed to assist in identifying & moderating content that may violate community guidelines/legal regulations, it may not be perfect.